

Wetime Identity Calibration System

Complete Flutter Development Guide for Junior Developers

PART 1: UNDERSTANDING THE PROBLEM

Why Does This Feature Exist?

Imagine you download a travel app. The first thing it shows you is a boring form:

- "Select your travel group"
- "Enter your budget"
- "Choose your preferences"

You would close the app immediately. This is called "friction" - anything that feels like work makes users leave.

But here's the problem: **We NEED this information** to give good recommendations. Without knowing if you're traveling solo or with kids, we can't filter properly.

The Solution: Make Data Collection Feel Like a Game

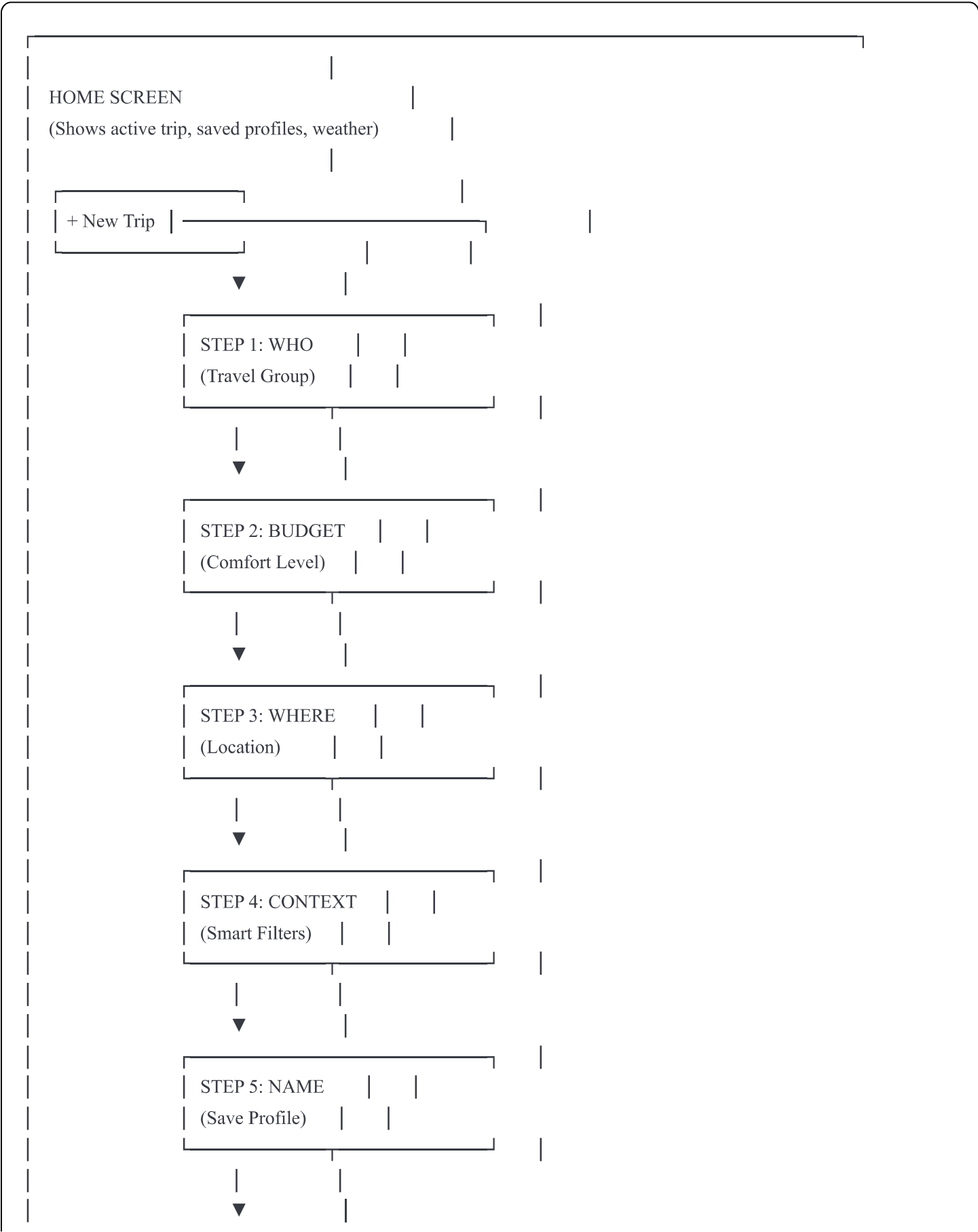
Instead of a form, we create an **"Identity Calibration"** experience:

- It looks like a personality quiz (fun!)
- Each step gives visual feedback (rewarding!)
- You see immediate value at the end (satisfying!)

The user thinks: "Cool, I'm discovering my travel identity" **What actually happens:** We collect: group, budget, location, and preferences

PART 2: THE COMPLETE USER JOURNEY

Screen Flow Overview



PROCESSING

(Loading Animation)



BACK TO HOME

(With new profile)

PART 3: EACH SCREEN EXPLAINED

Screen 1: HOME SCREEN

What the user sees:

Wetime

[Avatar]



14°C in Brussels

Light rain expected

[Indoor +40%]

ACTIVE TRIP



Solo Weekend

Brussels • Budget

234 89 12

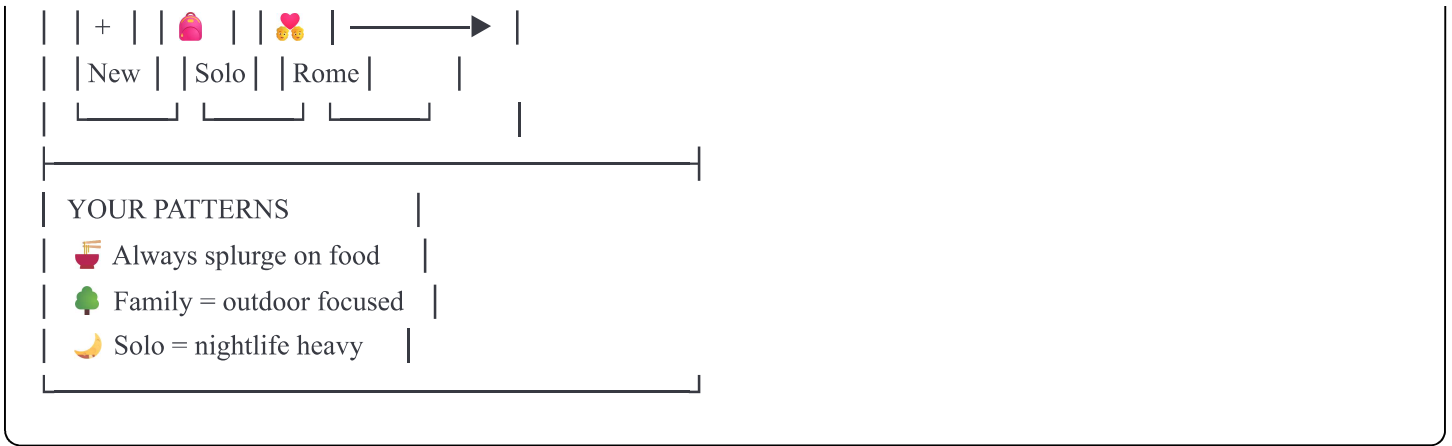
Matches Weather Near

[Explore]

[Edit]

SAVED PROFILES

[+ New]



What this screen does:

1. Weather Banner (Top)

- Shows current weather for active trip location
- Displays how weather AFFECTS recommendations
- Example: "Raining → Indoor spots prioritized"

2. Active Trip Card (Middle)

- Shows which profile is currently being used
- Displays stats: how many matches, weather-filtered, nearby
- Two buttons: "Explore" (go to recommendations) and "Edit"

3. Saved Profiles (Horizontal Scroll)

- Shows all saved trip profiles as cards
- User can tap to switch active profile
- "+ New" card starts calibration flow

4. Pattern Insights (Bottom)

- Shows what the AI has learned across ALL profiles
- Example: "You always pick food places even on budget trips"
- This data comes from analyzing multiple saved profiles

Data needed for this screen:

- activeTrip: The currently selected profile
- savedProfiles: List of all saved profiles

- weather: Current weather data for active location
- patterns: AI-generated insights (can be hardcoded initially)

Screen 2: STEP 1 - TRAVEL GROUP

What the user sees:

Step 1/5

IDENTITY

Who's on this trip?

This shapes pace & places



Solo



Couple

Freedom

Romance



Family



Friends

Kid-focus

Squad

Continue

How it works:

1. **Progress Bar** - Shows step 1 of 5 (20% filled)
2. **Question Section**
 - Small label: "IDENTITY" (category)

- Big title: "Who's on this trip?"
- Subtitle: Explains why we're asking

3. Option Cards (2x2 Grid)

- Each card has: Emoji, Label, Short description
- Only ONE can be selected at a time
- When selected: border changes color, checkmark appears

4. Continue Button

- DISABLED (grayed out) until user selects an option
- ENABLED (blue/accent color) after selection
- Tapping goes to Step 2

User interaction:

1. User taps "Solo" card
2. Card gets highlighted (border + background change)
3. Other cards stay normal
4. Continue button becomes active
5. User taps Continue
6. Navigate to Step 2, save selection: group = "solo"

Data collected:

group: "solo" | "couple" | "family" | "friends"

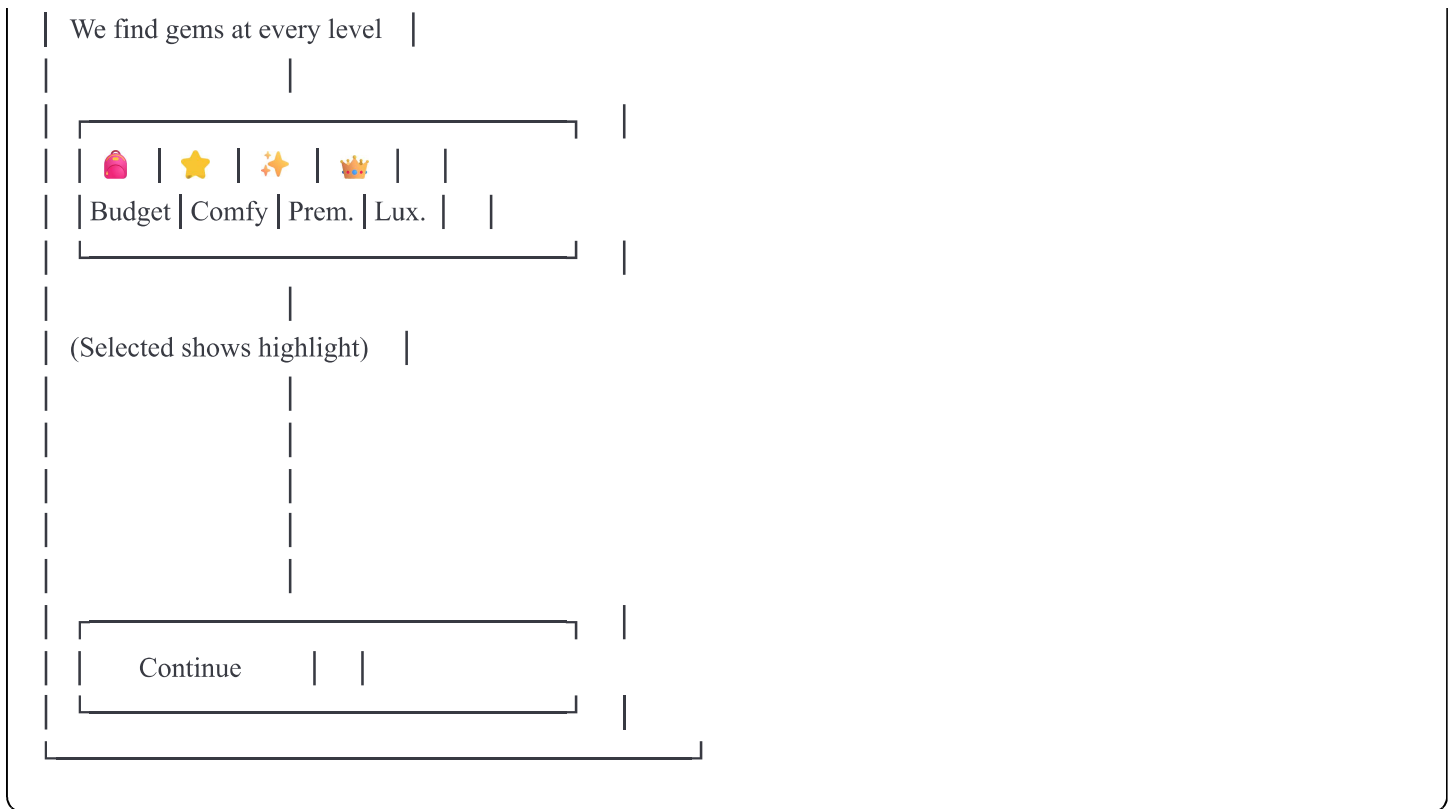
Screen 3: STEP 2 - BUDGET

What the user sees:

[←] Step 2/5

RESOURCES

Your comfort zone?



How it works:

1. **Segmented Control** (not cards this time)
 - 4 options in a horizontal row
 - Looks like a toggle/tab bar
 - Selected option is highlighted (filled background)
2. **Why different from Step 1?**
 - Budget is a "scale" (low to high)
 - Horizontal layout shows progression
 - Feels different from previous step (variety keeps user engaged)

User interaction:

1. User taps "Comfort" segment
2. That segment fills with accent color
3. Continue button activates
4. Navigate to Step 3

Data collected:

budget: "budget" | "mid" | "premium" | "luxury"

Screen 4: STEP 3 - LOCATION

What the user sees:

Step 3/5

DESTINATION

Where to?

We'll tune into local gems

Search city...

BE Brussels, Belgium

ES Barcelona, Spain

FR Paris, France

NL Amsterdam, Neth.

Use current location

Continue

How it works:

1. Search Input

- Text field with search icon
- As user types, suggestions appear below
- Suggestions filter based on input

2. Suggestions Dropdown

- Shows city + country + flag emoji
- Tapping a suggestion fills the input
- Dropdown closes after selection

3. GPS Button

- Dashed border (looks different from primary actions)
- Tapping requests device location
- Auto-fills with detected city

User interaction:

Option A - Type search:

1. User taps search field
2. Keyboard opens
3. User types "Bar"
4. Suggestions show: Barcelona, Bari, etc.
5. User taps "Barcelona"
6. Input shows "Barcelona"
7. Continue activates

Option B - Use GPS:

1. User taps "Use current location"
2. App requests location permission
3. App detects coordinates
4. App converts to city name (reverse geocoding)
5. Input shows "Brussels (Current)"
6. Continue activates

Data collected:

location: "Barcelona"
country: "Spain"
coordinates: { lat: 41.3851, lng: 2.1734 } (optional, from GPS)

Important for Flutter:

- You need location permission handling
- You need a city database OR an API (Google Places, etc.)
- GPS button needs to handle permission denied gracefully

Screen 5: STEP 4 - CONTEXT SETTINGS

What the user sees:

Step 4/5

SMART FILTERS

Context awareness

Let the app adapt to reality

Weather Filtering

[●]

Hide outdoor when rain

Time Awareness

[●]

Bars at night, café AM

Crowd Avoidance

[○]

Skip busy hotspots

Live Availability

[●]

Only show open spots

Continue

How it works:

1. Toggle Rows

- Each row is a setting with on/off toggle
- Icon + Title + Description + Switch
- Some are ON by default, some OFF

2. No validation needed

- Continue is always active
- User can skip without changing anything
- Defaults are sensible

What each toggle means:

Toggle	When ON	When OFF
Weather	Don't show parks when raining	Show everything
Time	Show bars at night, cafés morning	Show everything
Crowds	Deprioritize busy places	Show popular spots
Availability	Only show bookable/open	Show all

Data collected:

```
context: {  
  weather: true,  
  time: true,  
  crowds: false,  
  availability: true  
}
```

Screen 6: STEP 5 - NAME YOUR TRIP

What the user sees:

Step 5/5

IDENTITY

Name this trip

Save it for future use

Solo Weekend Explorer

Suggestions:

Solo Barca

Budget Trip

Barca Advent.

Create Trip Profile

How it works:

- Text Input**
 - User types a custom name
 - OR taps a suggestion chip
- Suggestion Chips**
 - Auto-generated based on previous selections
 - Example: "Solo" + "Barcelona" = "Solo Barcelona"

- Tapping chip fills the input

3. Create Button

- Only active when input has text
- Tapping starts processing animation

Generating suggestions (logic):

```
groupNames = {
  solo: ["Solo", "Me Time"],
  couple: ["Romantic", "Duo"],
  family: ["Family", "Kids"],
  friends: ["Squad", "Crew"]
}

budgetNames = {
  budget: ["Budget", "Savvy"],
  mid: ["Comfort"],
  premium: ["Premium"],
  luxury: ["Luxury"]
}

// Combine to create suggestions:
Suggestion 1: groupNames[group][0] + " " + location
Suggestion 2: budgetNames[budget][0] + " " + groupNames[group][1]
Suggestion 3: location + " Adventure"
```

Data collected:

name: "Solo Weekend Explorer"

Screen 7: PROCESSING

What the user sees:





(pulsing animation)

Building your trip profile

Connecting to real-time data

✓ Profile parameters set

✓ Fetching weather data

○ Matching recommendations

○ Applying context filters

How it works:

1. Pulsing Animation

- Circle that grows and fades
- Creates sense of activity

2. Step List

- Shows what's happening
- Each step gets checkmark after delay
- Creates anticipation

Animation Timing:

0.0s - Show screen, start pulse

0.5s - Step 1 gets checkmark ✓

1.2s - Step 2 gets checkmark ✓

1.9s - Step 3 gets checkmark ✓

2.6s - Step 4 gets checkmark ✓

3.0s - Navigate to Home screen

What actually happens during processing:

1. Save profile to local storage

2. Make profile the active profile
3. Fetch weather for selected location (API call)
4. Pre-fetch some recommendations (optional)
5. Navigate to Home

PART 4: THE DATA MODEL

Profile Object Structure

This is what ONE trip profile looks like:

```
TripProfile {
  id: String,           // Unique ID (timestamp or UUID)

  // Core calibration data
  group: String,        // "solo" | "couple" | "family" | "friends"
  budget: String,       // "budget" | "mid" | "premium" | "luxury"
  location: String,     // "Barcelona"
  country: String,      // "Spain"

  // Context settings
  context: {
    weather: Boolean,   // Filter by weather
    time: Boolean,      // Filter by time of day
    crowds: Boolean,    // Avoid crowded places
    availability: Boolean // Only show available
  },

  // Meta
  name: String,         // "Solo Weekend Explorer"
  icon: String,         // "🏠" (based on group)
  createdAt: DateTime,  // When created
  useCount: Integer     // How many times activated
}
```

App State Structure

The entire app state:

```
AppState {  
  // Current calibration in progress (temporary)  
  currentCalibration: {  
    group: String?,  
    budget: String?,  
    location: String?,  
    country: String?,  
    context: {...},  
    name: String?  
  },  
  
  // Persistent data  
  activeProfile: TripProfile?, // Currently active  
  savedProfiles: List<TripProfile>, // All saved  
  
  // Live data  
  weather: {  
    temp: Integer,  
    condition: String,  
    icon: String,  
    effects: List<String>  
  }  
}
```

PART 5: STATE MANAGEMENT EXPLAINED

What is State?

"State" is data that can change while the app is running.

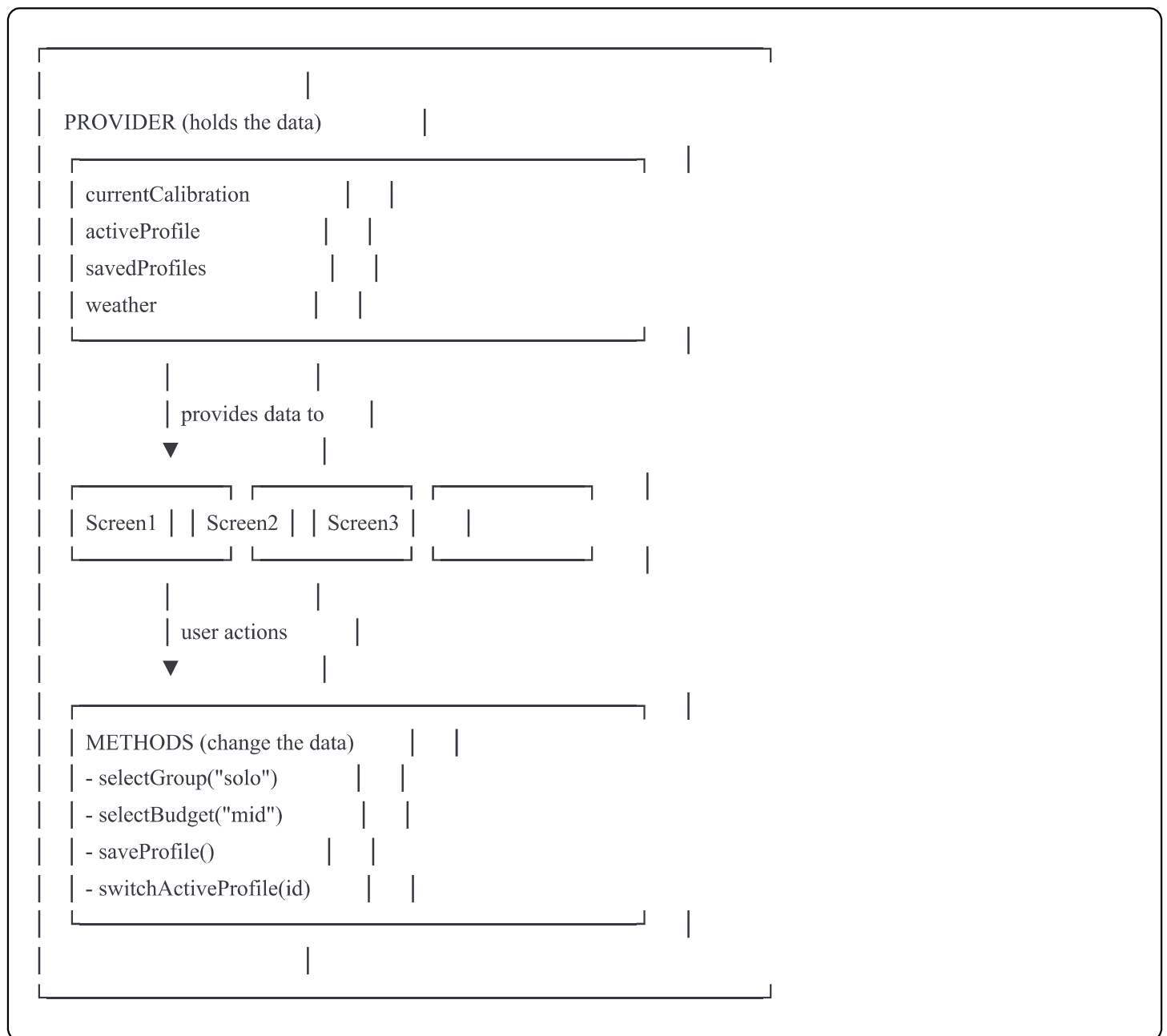
Examples of state in this app:

- Which step of calibration are we on?
- What has the user selected so far?
- Which profile is active?
- What's the current weather?

Recommended Approach for Flutter

Use **Provider** or **Riverpod** (Riverpod is newer and cleaner).

Simple Mental Model:



How screens interact with state:

Reading data:

Screen needs to show the selected group?

→ Read from provider: `currentCalibration.group`

→ If "solo", highlight the Solo card

Writing data:

User taps "Couple" card
→ Call provider method: `selectGroup("couple")`
→ Provider updates: `currentCalibration.group = "couple"`
→ Screen automatically rebuilds with new data

PART 6: NAVIGATION

Flutter Navigation Options

Option 1: Named Routes (Simple)

Routes:

`/home`
`/calibrate/group` (Step 1)
`/calibrate/budget` (Step 2)
`/calibrate/location` (Step 3)
`/calibrate/context` (Step 4)
`/calibrate/name` (Step 5)
`/calibrate/processing`

Option 2: Go Router (Recommended)

Better for complex apps, handles deep links, back button properly.

Navigation Flow:

HOME —[+ New Trip]→ STEP 1
|
[Continue]
▼
STEP 2
|
[Continue]
▼
STEP 3
|
[Continue]



Back Button Behavior:

- Each step can go back to previous step
- Back from Step 1 goes to Home
- Calibration data is preserved when going back
- Only cleared when reaching Home

PART 7: LOCAL STORAGE

What needs to be saved permanently?

When user closes app and reopens:

- ☒ Saved profiles should still exist
- ☒ Active profile should be remembered
- ☒ Current calibration in progress can be lost

Flutter Options:

Option 1: SharedPreferences

- Simple key-value storage

- Good for small data
- Store JSON string of profiles

Option 2: Hive (Recommended)

- Fast local database
- Better for lists of objects
- Easy to use with Flutter

Option 3: SQLite

- Full database
- Overkill for this use case

What to save:

Key: "saved_profiles"

Value: JSON list of TripProfile objects

Key: "active_profile_id"

Value: String ID of active profile

When to save:

- After creating new profile → Save profiles list
- After switching active profile → Save active ID
- After editing profile → Save profiles list
- After deleting profile → Save profiles list

PART 8: API INTEGRATIONS

APIs You Need:

1. Weather API (Required)

Purpose: Get current weather for selected city

Recommended: OpenWeatherMap (free tier available)

When to call:

- When user selects a location in calibration
- When loading Home screen (for active profile location)
- Periodically refresh (every 30 min)

Data you need from API:

```
{  
  temperature: 14,  
  condition: "rain" | "sunny" | "cloudy" | "snow",  
  description: "Light rain"  
}
```

Transform to display:

```
condition "rain" → icon "🌧️", effects ["Indoor priority", "Museums +40%"]  
condition "sunny" → icon "☀️", effects ["Beach day!", "Outdoor +60%"]
```

2. City Search API (Required)

Purpose: Search cities as user types

Options:

- Google Places API (paid but best)
- OpenCage Geocoding (cheaper)
- Local database of major cities (free, limited)

When to call:

- As user types in location field
- Debounce: wait 300ms after last keystroke before calling

3. Reverse Geocoding API (For GPS feature)

Purpose: Convert GPS coordinates to city name

When to call:

- When user taps "Use current location"
- After getting device coordinates

Flow:

1. Get device location: lat=50.8503, lng=4.3517
2. Call reverse geocoding API
3. Get response: "Brussels, Belgium"
4. Fill in location field

4. Your Recommendation API (Later)

Purpose: Get personalized recommendations

When to call:

- When user taps "Explore" on Home screen
 - Pass the active profile data as filters
-

PART 9: FLUTTER PROJECT STRUCTURE

Recommended Folder Structure:

```
lib/  
├── main.dart  
│  
├── models/  
│   ├── trip_profile.dart    // TripProfile class  
│   └── weather.dart        // Weather class  
│  
├── providers/  
│   ├── calibration_provider.dart // Current calibration state  
│   ├── profiles_provider.dart   // Saved profiles  
│   └── weather_provider.dart    // Weather data  
│  
├── screens/  
│   └── home/
```

```
| | └─ home_screen.dart
| |
| └─ calibration/
|   └─ step1_group_screen.dart
|   └─ step2_budget_screen.dart
|   └─ step3_location_screen.dart
|   └─ step4_context_screen.dart
|   └─ step5_name_screen.dart
|   └─ processing_screen.dart
|
└─ widgets/
  └─ option_card.dart      // Selectable card widget
  └─ budget_selector.dart  // Budget toggle bar
  └─ location_input.dart   // Search with suggestions
  └─ toggle_row.dart       // Setting with switch
  └─ profile_card.dart     // Saved profile card
  └─ weather_banner.dart   // Weather display
  └─ progress_header.dart  // Step progress bar
|
└─ services/
  └─ weather_service.dart  // Weather API calls
  └─ location_service.dart // GPS + city search
  └─ storage_service.dart  // Local storage
|
└─ theme/
  └─ app_theme.dart        // Colors, fonts, etc.
```

PART 10: DESIGN SPECIFICATIONS

Clean Design System

Colors (Light Theme):

Background Primary: #FFFFFF (white)

Background Secondary: #F8F9FA (light gray)

Background Tertiary: #F1F3F4 (medium gray)

Text Primary: #1A1A1A (almost black)

Text Secondary: #5F6368 (dark gray)

Text Tertiary: #9AA0A6 (medium gray)

Accent: #1A73E8 (Google blue)
Accent Light: #E8F0FE (light blue)

Success: #34A853 (green)
Warning: #FBBC04 (yellow/orange)

Border: #E8EAED (light border)

Typography:

Font Family: Inter (or SF Pro on iOS)

Sizes:

- Page Title: 28px, Bold (700)
- Section Title: 14px, Semibold (600), Uppercase
- Card Title: 16px, Semibold (600)
- Body: 16px, Regular (400)
- Caption: 14px, Regular (400)
- Small: 12px, Medium (500)

Spacing:

Page Padding: 24px
Card Padding: 20px
Section Gap: 24px
Element Gap: 12px

Border Radius:

Small (buttons, inputs): 8px
Medium (cards): 12px
Large (modals): 16px
Full (pills, avatars): 100px

Shadows:

Small: 0 1px 2px rgba(0,0,0,0.05)
Medium: 0 4px 12px rgba(0,0,0,0.08)
Large: 0 8px 24px rgba(0,0,0,0.12)

PART 11: DEVELOPMENT ROADMAP

Phase 1: Foundation (Week 1)

Day 1-2: Project Setup

- ☐ Create new Flutter project
- ☐ Set up folder structure
- ☐ Install dependencies (provider/riverpod, go_router, hive)
- ☐ Create theme file with colors, fonts
- ☐ Create basic app shell with navigation

Day 3-4: Data Models

- ☐ Create TripProfile model class
- ☐ Create Weather model class
- ☐ Create CalibrationState model class
- ☐ Set up Hive boxes for storage
- ☐ Test saving/loading profiles

Day 5: State Management

- ☐ Set up Provider/Riverpod
 - ☐ Create CalibrationProvider
 - ☐ Create ProfilesProvider
 - ☐ Test state updates
-

Phase 2: Calibration Flow (Week 2)

Day 1: Step 1 - Group Selection

- ☐ Create screen layout
- ☐ Build OptionCard widget
- ☐ Connect to state
- ☐ Handle selection
- ☐ Enable/disable continue button

Day 2: Step 2 - Budget Selection

- ☐ Create screen layout
- ☐ Build BudgetSelector widget
- ☐ Connect to state
- ☐ Handle selection

Day 3: Step 3 - Location

- ☐ Create screen layout
- ☐ Build LocationInput widget
- ☐ Add static suggestions (no API yet)
- ☐ Handle selection
- ☐ Add GPS button (permission handling)

Day 4: Step 4 - Context Toggles

- ☐ Create screen layout
- ☐ Build ToggleRow widget
- ☐ Connect to state
- ☐ Set default values

Day 5: Step 5 - Name + Processing

- ☐ Create name input screen
 - ☐ Generate suggestion chips
 - ☐ Create processing screen
 - ☐ Add loading animation
 - ☐ Save profile on complete
-

Phase 3: Home Screen (Week 3)

Day 1-2: Active Trip Card

- ☐ Create card layout
- ☐ Display active profile data
- ☐ Add stat counters (hardcoded for now)
- ☐ Add Explore and Edit buttons

Day 3: Saved Profiles

- ☐ Create horizontal scroll list
- ☐ Build ProfileCard widget
- ☐ Handle tap to switch active
- ☐ Add "+ New" card

Day 4: Weather Banner

- ☐ Create banner layout
- ☐ Add static weather data
- ☐ Display effects based on condition

Day 5: Pattern Insights

- ☐ Create insights section
 - ☐ Add static insight items
 - ☐ Style appropriately
-

Phase 4: API Integration (Week 4)

Day 1-2: Weather API

- ☐ Sign up for OpenWeatherMap
- ☐ Create WeatherService class
- ☐ Implement API call
- ☐ Parse response
- ☐ Connect to weather banner

Day 3-4: Location Search

- ☐ Choose API (or use local database)
- ☐ Create LocationService class
- ☐ Implement search-as-you-type
- ☐ Add debouncing
- ☐ Implement reverse geocoding for GPS

Day 5: Polish

- ☐ Add loading states
- ☐ Add error handling

- ☐ Test edge cases
 - ☐ Fix bugs
-

Phase 5: Polish & Testing (Week 5)

Day 1-2: Animations

- ☐ Add page transitions
- ☐ Add card selection animations
- ☐ Add button press feedback
- ☐ Add progress bar animation

Day 3: Quick Switch Modal

- ☐ Create modal overlay
- ☐ Display saved profiles
- ☐ Handle selection
- ☐ Animate open/close

Day 4-5: Testing

- ☐ Test full calibration flow
 - ☐ Test profile saving/loading
 - ☐ Test weather updates
 - ☐ Test GPS functionality
 - ☐ Test on different screen sizes
 - ☐ Fix any remaining bugs
-

PART 12: COMMON MISTAKES TO AVOID

Mistake 1: Not handling empty states

- ✗ App crashes when no profiles saved
- ✓ Show "Create your first trip" message

Mistake 2: Not handling loading states

- ✗ Screen is blank while fetching weather
- ✓ Show skeleton loader or spinner

Mistake 3: Not handling errors

- ✗ App crashes when API fails
- ✓ Show error message, allow retry

Mistake 4: Not debouncing search

- ✗ API called on every keystroke
- ✓ Wait 300ms after user stops typing

Mistake 5: Losing calibration on back

- ✗ Going back clears previous selections
- ✓ Keep selections until user cancels or completes

Mistake 6: Not handling permissions

- ✗ GPS button crashes without permission
- ✓ Request permission, handle denial gracefully

PART 13: TESTING CHECKLIST

Before saying "it's done", verify:

Calibration Flow

- ☐ Can complete all 5 steps
- ☐ Can go back without losing data
- ☐ Continue button only active after selection
- ☐ Profile saves correctly

- ☐ Navigates to home after processing

Home Screen

- ☐ Shows active profile correctly
- ☐ Saved profiles appear in scroll
- ☐ Can switch active profile
- ☐ Weather displays for active location
- ☐ "+ New" starts calibration

Persistence

- ☐ Profiles survive app restart
- ☐ Active profile remembered
- ☐ Works after phone restart

Edge Cases

- ☐ Works with 0 saved profiles
 - ☐ Works with 10+ saved profiles
 - ☐ Works offline (graceful degradation)
 - ☐ Works on small screens
 - ☐ Works on tablets
-

SUMMARY

What You're Building:

A 5-step "identity calibration" flow that collects travel preferences in an engaging way, saves them as reusable profiles, and adapts recommendations based on real-world context (weather, time).

Key Screens:

1. **Home** - Dashboard with active trip, saved profiles, weather
2. **Step 1** - Travel group (solo/couple/family/friends)
3. **Step 2** - Budget level (4 tiers)
4. **Step 3** - Location (search + GPS)
5. **Step 4** - Context toggles (weather, time, crowds, availability)

6. **Step 5** - Name your trip
7. **Processing** - Loading animation, then redirect home

Key Data:

- TripProfile object with all selections
- Saved to local storage (Hive)
- Weather fetched from API
- State managed with Provider/Riverpod

Key Principle:

Make it feel like discovery, not a form. Big visual cards, satisfying animations, immediate feedback.

Good luck building! 🚀